

React Compiler Installation: Hinglish Explanation

Ye guide batati hai ki apne React application mein **React Compiler ko install, configure aur verify** kaise karna hai.

React Compiler kya karta hai?

React Compiler aapke React components aur calculations ko automatically optimize karta hai. Iska primary benefit ye hai ki kai situations mein aapko manually:

- useMemo
- useCallback
- React.memo

jaise optimization tools use nahi karne padte.

Compiler build ke time aapka code analyze karke required memoization logic automatically add kar deta hai.

1. Prerequisites

React Compiler **React 19 ke saath best kaam karta hai**, lekin ye React 17 aur React 18 ko bhi support karta hai.

Agar project React 17 ya 18 use karta hai, to compiler ki **target configuration** ki zarurat pad sakti hai.

2. React Compiler install karna

React Compiler ka package hai:

```
1 babel-plugin-react-compiler
```

Isse devDependency ke roop mein install kiya jata hai, kyunki ye development aur build process ke dauran code compile karta hai. Application ke runtime dependency ke roop mein generally iski zarurat nahi hoti.

npm

```
1 npm install -D babel-plugin-react-compiler@latest
```

Yarn

```
1 yarn add -D babel-plugin-react-compiler@latest
```

pnpm

```
1 pnpm install -D babel-plugin-react-compiler@latest
```

Yahan:

- -D ka matlab package ko devDependencies mein save karna hai.
 - @latest package ka latest available version install karta hai.
-

3. Basic Setup

React Compiler ko default situation mein additional configuration ki zarurat nahi hoti. Bas apne build tool ke Babel pipeline mein compiler plugin add karna hota hai.

Special configuration tab chahiye ho sakti hai jab:

- Aap React 17 ya 18 target kar rahe ho.
- Existing project mein compiler ko gradually introduce karna ho.
- Kuch components ko compilation se exclude karna ho.
- Compiler ke behavior ko customize karna ho.

Sabse important rule

babel-plugin-react-compiler ko Babel plugin pipeline mein **sabse pehle run hona chahiye**.

Compiler ko analysis ke liye original source code ki information chahiye. Agar koi aur Babel plugin pehle code transform kar dega, to React Compiler proper analysis nahi kar payega.

4. Babel ke saath setup

Apni babel.config.js file create ya update karo:

```
1 module.exports = {  
2   plugins: [  
3     'babel-plugin-react-compiler', // must run first!  
4     // ...other plugins  
5   ],  
6   // ...other config  
7 };
```

Dhyan rahe ki compiler plugin array mein first position par hai:

```
1 plugins: [  
2   'babel-plugin-react-compiler',
```

```
3 ]
```

Agar aur Babel plugins hain, to unhe iske baad rakho.

5. Vite ke saath setup

Vite ka setup is baat par depend karta hai ki aap `@vitejs/plugin-react` ka kaunsa version use kar rahe ho.

Vite React plugin 6.0.0 ya newer

Agar aap `@vitejs/plugin-react` version 6.0.0 ya uske baad ka version use kar rahe ho, to `reactCompilerPreset` use kar sakte ho.

Pehle Babel integration package install karo:

```
1 npm install -D @rolldown/plugin-babel
```

Phir vite.config.js configure karo:

```
1 import { defineConfig } from 'vite';
2 import react, { reactCompilerPreset } from '@vitejs/plugin-react';
3 import babel from '@rolldown/plugin-babel';
4
5 export default defineConfig({
6   plugins: [
7     react(),
8     babel({
9       presets: [reactCompilerPreset()],
10     }),
11   ],
12 });
```

Is configuration mein kya ho raha hai?

```
1 react()
```

Ye standard Vite React plugin activate karta hai.

```
1 babel({
2   presets: [reactCompilerPreset()],
3 })
```

Ye Babel processing ke andar React Compiler ka official preset add karta hai.

Older @vitejs/plugin-react version

Purane versions mein react() ke andar inline babel option available tha:

```
1 import { defineConfig } from 'vite';
2 import react from '@vitejs/plugin-react';
3
4 export default defineConfig({
5   plugins: [
6     react({
7       babel: {
8         plugins: ['babel-plugin-react-compiler'],
9       },
10    }),
11  ],
12 });
```

Lekin @vitejs/plugin-react@6.0.0 mein ye inline Babel option remove kar diya gaya hai.

Isliye ye method **sirf older versions** ke liye hai.

Vite mein Babel plugin directly use karna

Preset ke alternative ke roop mein direct plugin bhi configure kar sakte ho:

```
1 import { defineConfig } from 'vite';
2 import react from '@vitejs/plugin-react';
3 import babel from '@rolldown/plugin-babel';
4
5 export default defineConfig({
6   plugins: [
7     react(),
8     babel({
9       plugins: ['babel-plugin-react-compiler'],
10    }),
11  ],
12 });
```

Is approach mein reactCompilerPreset() ki jagah compiler ka Babel plugin directly diya gaya hai.

6. Next.js ke saath setup

Next.js mein React Compiler ka configuration Next.js ki apni configuration ke through hota hai.

Iske liye page ne direct Next.js documentation refer karne ko kaha hai. Uska external link neeche diya gaya hai.

7. React Router ke saath setup

React Router ke Vite setup mein vite-plugin-babel install karo:

```
1 npm install vite-plugin-babel
```

Phir vite.config.js mein compiler configure karo:

```
1 import { defineConfig } from "vite";
2 import babel from "vite-plugin-babel";
3 import { reactRouter } from "@react-router/dev/vite";
4
5 const ReactCompilerConfig = {
6   /* compiler options */
7 };
8
9 export default defineConfig({
10   plugins: [
11     reactRouter(),
12
13     babel({
14       filter: /\.([jt]sx?)$/,
15       babelConfig: {
16         presets: ["@babel/preset-typescript"],
17         plugins: [
18           ["babel-plugin-react-compiler",
19             ReactCompilerConfig],
20         ],
21       },
22     }),
23   ]});
```

Important parts

File filter

```
1 filter: /\.([jt]sx?$/
```

Ye filter compiler ko in file types par apply karta hai:

- .js
- .jsx
- .ts
- .tsx

TypeScript preset

```
1 presets: ["@babel/preset-typescript"]
```

Agar aap TypeScript use karte ho, tab ye preset include karo.

Compiler configuration

```
1 [  
2   "babel-plugin-react-compiler",  
3   ReactCompilerConfig,  
4 ]
```

Yahan plugin ke saath custom compiler options pass kiye ja sakte hain.

8. Webpack ke saath setup

Webpack ke liye ek **community-maintained Webpack loader** available hai.

Dhyan rahe, page ise community loader bol raha hai. Isliye ise React ka official Webpack integration assume nahi karna chahiye.

9. Expo ke saath setup

Expo applications ke liye React Compiler enable aur use karne ke instructions Expo ki documentation mein diye gaye hain.

10. Metro aur React Native

React Native ka Metro bundler Babel use karta hai.

Isliye React Native aur Metro ke liye basic Babel setup follow karna hai:

```
1  plugins: [  
2    'babel-plugin-react-compiler',  
3  ]
```

Compiler plugin ko yahan bhi Babel pipeline mein first rakhna important hai.

11. Rspack aur Rsbuild

Rspack aur Rsbuild ke apne React Compiler integrations hain.

- Rspack users ko Rspack documentation follow karni hai.
- Rsbuild users ko Rsbuild documentation follow karni hai.

Dono external documentation links neeche diye gaye hain.

12. ESLint Integration

React Compiler ke saath ESLint rule milta hai jo identify karta hai ki kaunsa code safely optimize nahi ho sakta.

Latest React Hooks ESLint plugin install karo:

```
1  npm install -D eslint-plugin-react-hooks@latest
```

Compiler rules recommended-latest preset mein available hain.

ESLint rule kya check karta hai?

Ye rule:

- Rules of React violations identify karta hai.
- Batata hai ki kaunsa component optimize nahi ho sakta.
- Problem fix karne ke liye helpful error messages deta hai.

ESLint error ka meaning

Agar compiler-related ESLint error aata hai, to iska matlab poori application compilation fail hona zaruri nahi hai.

Compiler us particular problematic:

- component, ya
- custom hook

ko optimize karna skip kar sakta hai, aur baaki compatible code ko optimize karta rahega.

Isliye saare violations ko ek saath fix karna compulsory nahi hai. Aap gradually fixes karke optimized components ki sankhya badha sakte ho.

13. Setup verify kaise karein?

Installation ke baad check karna zaruri hai ki compiler actual mein run ho raha hai.

Do main verification methods diye gaye hain:

1. React DevTools check karna
 2. Build output inspect karna
-

Method 1: React DevTools

React Developer Tools browser extension install karo. Uske baad:

1. Application ko development mode mein open karo.
2. Browser DevTools open karo.
3. React DevTools section mein jao.
4. Component names ke paas Memo 💡 badge dekho.

Agar component compiler se optimize hua hai, to uske paas kuch is tarah ka badge show hoga:

1 Memo 💡

Compiler working hone par:

- Optimized components ke paas Memo 💡 badge show hoga.
- Expensive calculations automatically memoize ho sakti hain.
- Manual useMemo ki zarurat nahi padti.

Ye nahi maana jana chahiye ki har component par badge zarur dikhega. Compiler sirf us code ko optimize karega jise wo safely optimize kar sakta hai.

Method 2: Build output check karna

Compiler-generated code mein automatic memoization logic nazar aa sakta hai:

```
1 import { c as _c } from "react/compiler-runtime";
2
```

```
3 export default function MyApp() {
4   const $ = _c(1);
5
6   let t0;
7
8   if ($[0] === Symbol.for("react.memo_cache_sentinel")) {
9     t0 = <div>Hello World</div>;
10    $[0] = t0;
11  } else {
12    t0 = $[0];
13  }
14
15  return t0;
16 }
```

Aapko manually aisa code likhna nahi hai. Ye compiler-generated output ka example hai.

Code kya kar raha hai?

```
1 const $ = _c(1);
```

Compiler memoization ke liye cache create ya access kar raha hai.

```
1 if ($[0] === Symbol.for("react.memo_cache_sentinel"))
```

Ye check karta hai ki cache position mein value pehle se stored hai ya nahi.

Agar value cache mein nahi hai:

```
1 t0 = <div>Hello World</div>;
2 $[0] = t0;
```

To value create karke cache mein store kar di jati hai.

Agar value already cached hai:

```
1 t0 = $[0];
```

To existing value reuse hoti hai.

Simple words mein, compiler automatically detect karke reusable output ko cache kar raha hai.

14. Kisi specific component ko compiler se exclude karna

Agar compilation ke baad koi component issue create kar raha hai, to temporarily "use no memo" directive use kar sakte ho:

```
1 function ProblematicComponent() {  
2   "use no memo";  
3  
4   // Component code  
5 }
```

Is directive ka meaning hai:

React Compiler is particular component ko optimize na kare.

Important point

Ye permanent solution nahi maana gaya hai.

Recommended approach:

1. Temporarily "use no memo" add karo.
2. Component ki actual underlying problem identify karo.
3. Problem fix karo.
4. Directive remove kar do.

Isse baaki application compiler optimizations use karti rahegi, jabki problematic component temporarily skip ho jayega.

Build-tool selection summary

Plain Babel project

```
1 plugins: [  
2   'babel-plugin-react-compiler',  
3 ]
```

Vite with @vitejs/plugin-react 6+

```
1 babel({  
2   presets: [reactCompilerPreset()],  
3 })
```

Older Vite React plugin

```
1  react({
2    babel: {
3      plugins: ['babel-plugin-react-compiler'],
4    },
5  })
```

React Router

vite-plugin-babel ke andar compiler ka Babel plugin configure karo.

React Native with Metro

Babel configuration wala approach follow karo.

Next.js, Expo, Rspack aur Rsbuild

In platforms ki respective documentation mein diya gaya integration follow karo.

Sabse important takeaways

1. React Compiler **React 19 ke saath best work karta hai**, lekin React 17 aur 18 bhi supported hain.
2. Package ko development dependency ke taur par install karo:

```
1  npm install -D babel-plugin-react-compiler@latest
```

1. Babel pipeline mein compiler plugin ko **first position** par rakho.
2. Vite configuration @vitejs/plugin-react ke version par depend karti hai.
3. ESLint violations ka matlab ye ho sakta hai ki sirf affected component ya hook optimize nahi hoga.
4. React DevTools mein Memo 💎 badge se optimization verify kar sakte ho.
5. Build output mein compiler-generated cache logic se bhi setup verify ho sakta hai.
6. Problematic component ko temporarily "use no memo" se opt out kar sakte ho.
7. Opt-out ke baad underlying issue fix karke directive remove karna chahiye.

External links used on the page

- [Next.js React Compiler configuration](#)
- [React Hooks ESLint plugin installation](#)
- <https://github.com/SukkaW/react-compiler-webpack>
- [Expo React Compiler guide](#)
- [Rspack React Compiler guide](#)

- [Rsbuild React Compiler guide](#)